Manual Testing – Interview Syllabus

Chapter 1: Software Testing Fundamentals

Topics

- Definition of software testing
- Purpose & objectives of testing
- Errors, defects, failures
- Verification vs validation
- QA vs QC
- Severity vs priority
- Principles of software testing
- Entry & exit criteria
- Role and qualities of a tester
- Testing vs debugging

Chapter 2: SDLC & STLC

Topics

- SDLC overview & importance
- SDLC models (Waterfall, V-Model, Agile, Spiral)
- Testing activities in each SDLC phase
- STLC overview
- STLC phases
    - Requirement analysis
    - Test planning
    - Test case design
    - Environment setup
    - Test execution
    - Defect tracking
    - Test closure
- SDLC vs STLC

Chapter 3: Test Design Techniques & Test Case Development

Topics

- Test design fundamentals
- Test scenarios vs test cases

- Test case structure & writing
- Test design techniques
    - Equivalence Partitioning
    - Boundary Value Analysis
    - Decision Table Testing
    - State Transition Testing
    - Use Case Testing
    - Error Guessing
- Positive & negative testing
- Test coverage & traceability

Chapter 4: Types of Testing

Topics

- Functional testing
    - Smoke testing
    - Sanity testing
    - Regression testing
    - Retesting
    - Integration testing
    - System testing
    - User Acceptance Testing (UAT)
- Non-functional testing (overview)
    - Performance
    - Usability
    - Security
    - Compatibility
- Alpha & Beta testing
- Exploratory, Ad-hoc, Monkey testing

Chapter 5: Test Cases, Test Scenarios & Documentation

Topics

- Test scenarios creation
- Test case writing & components
- Positive, negative & boundary test cases
- Test case review & maintenance
- Test data preparation
- Requirement Traceability Matrix (RTM)
- Test documentation
    - Test plan
    - Test strategy
    - Test execution report
    - Test summary report

- Change & version management

Chapter 6: Defect Life Cycle & Bug Management

Topics

- Defect fundamentals
- Defect life cycle & statuses
- Severity vs priority (real-time mapping)
- Defect reporting & reproducibility
- Defect triage process
- Retesting & regression
- Defect leakage & escaped defects
- Defect metrics
    - Defect density
    - Defect aging
    - Defect slippage

Chapter 7: Agile & Scrum for Testers

Topics

- Agile principles & values
- Scrum framework
- Scrum roles & responsibilities
- Scrum artifacts
    - Product backlog
    - Sprint backlog
    - User stories
    - Story points
    - DoR & DoD
- Scrum ceremonies
- Agile testing practices
- Defect handling in Agile
- Agile metrics (velocity, burn-down)

Chapter 8: Web Application Testing Basics

Topics

- Web application fundamentals
- Client-server architecture
- HTTP vs HTTPS

- HTTP methods & status codes
- Cookies, sessions, storage
- Web forms & validations
- Browser & compatibility testing
- Basic security testing
- Performance & network basics

Chapter 9: Database & SQL Basics for Testers

Topics

- Database fundamentals
- Tables, keys & schema
- SQL basics (SELECT, WHERE, GROUP BY)
- Joins (INNER, LEFT, RIGHT)
- Constraints & indexes
- Transactions & ACID
- UI vs DB validation
- Database testing scenarios

Chapter 10: Test Environment, Build & Release Testing

Topics

- Test environments & types
- Environment readiness & issues
- Builds vs releases
- Build verification testing (BVT)
- Smoke & sanity testing for builds
- Release testing
- Go / No-Go decisions
- Deployment & production validation
- Post-release testing
- Risk communication & release sign-off

Table of Contents

Chapter 1 – Software Testing Fundamentals

Section 1: Introduction to Software Testing

1.1 Software Quality – The Foundation

Before defining software testing, we must understand **software quality**.

Software quality is the degree to which a software product:

- Meets stated and implied requirements
- Satisfies user needs
- Performs reliably under expected conditions

Quality is **not luxury**. It is survival.
In organizations, quality failures translate into:

- Financial loss
- Customer dissatisfaction
- Brand damage
- Legal and compliance risks

Testing exists to **protect quality**, not to decorate the product.

1.2 Evolution of Software Testing

Software testing did not begin as a formal discipline.

Early Phase

- Developers wrote code and tested it themselves
- Testing was synonymous with debugging
- Focus was only on “making it work”

Growth Phase

- Software size increased
- Failures became expensive
- Dedicated testing teams emerged
- Structured test cases and defect tracking started

Modern Phase

- Testing is involved from requirements stage
- Quality is shared across teams
- Risk-based and continuous testing models
- Testing supports business goals, not just code correctness

This evolution proves one thing:

Testing moved from **afterthought** to **core engineering activity**.

1.3 Common Misconceptions About Testing

Many failures happen not because testing is weak, but because it is **misunderstood**.

Common misconceptions include:

- Testing is only about finding bugs
- Testing starts after development
- If users didn't complain, quality is good
- Automation replaces manual testing completely
- Testers are responsible for quality alone

These misconceptions reduce testing to a **reactive task**, while in reality, testing is **preventive and analytical**.

1.4 Definition of Software Testing

Formal Definition

Software testing is the process of evaluating a software system or component to:

- Identify defects
- Verify compliance with requirements
- Validate that it meets user expectations
- Assess quality attributes such as reliability and usability

Interview-Friendly Definition

Software testing is a systematic activity performed to verify that the software behaves as expected and to identify defects before the product reaches the end user.

Key Words to Remember

- Systematic
- Evaluation
- Verification
- Validation
- Defect identification
- Quality assessment

These keywords are often **listened for** by interviewers.

1.5 Why Software Testing Is Necessary

Software is written by humans. Humans make mistakes.

Testing is necessary because:

- Requirements can be misunderstood
- Code can have logical flaws
- Integration points can break
- User behavior is unpredictable
- Real-world conditions differ from assumptions

Without testing:

- Defects reach production
- Fixing cost increases exponentially
- User trust erodes

Testing acts as a **risk control mechanism**.

1.6 Testing Is Not About Perfection

One important reality:

Testing cannot prove that software is defect-free.

Testing:

- Reduces risk
- Improves confidence
- Increases reliability

But **zero defects is not the goal**.
Acceptable risk is the goal.

This mindset is crucial for mature testers.

1.7 Relationship Between Testing and Business

Modern organizations test not only to ensure correctness, but to:

- Protect revenue
- Maintain compliance
- Improve user retention
- Support faster releases

A tester today indirectly safeguards:

- Customer experience
- Company reputation
- Operational stability

Testing has moved from **technical validation** to **business assurance**.

1.8 Summary of Section 1

- Software quality is the foundation of testing
- Testing evolved into a disciplined profession
- Testing is preventive, not just detective
- Software testing is a structured evaluation process
- Testing reduces risk, not guarantees perfection
- Testing supports both technical and business goals

Section 2: Purpose & Objectives of Software Testing

2.1 Why Organizations Invest in Testing

Software is not built for code completion; it is built for **use**.
Testing exists because **usage exposes risk**.
Organizations invest in testing to:

- Prevent costly production failures
- Protect customer trust
- Ensure regulatory and contractual compliance
- Control long-term maintenance cost
- Support predictable releases

Testing is a **business enabler**, not a cost center.

2.2 Purpose of Software Testing

The **purpose** of software testing is broader than defect detection.
At a high level, the purpose is to:

- Evaluate the quality of the software
- Identify gaps between expected and actual behavior
- Provide confidence to stakeholders
- Reduce risk before deployment

In short:

Testing provides **information** about the product so that informed decisions can be made.

2.3 Primary Objectives of Software Testing

The primary objectives focus on **correctness and compliance**.

1. **Defect Identification**
   To detect defects before they reach end users.
2. **Requirement Verification**
   To ensure the software meets documented requirements.
3. **Requirement Validation**
   To confirm the product behaves as users expect.
4. **Quality Assessment**
   To evaluate reliability, usability, performance, and stability.
5. **Risk Reduction**
   To minimize business, technical, and operational risks.

2.4 Secondary Objectives of Software Testing

Beyond defect detection, testing also aims to:

- Improve product design through early feedback
- Support release decisions
- Measure product readiness
- Build confidence among stakeholders
- Ensure maintainability and scalability

These objectives mature as organizations mature.

2.5 Preventive vs Detective Nature of Testing

Testing is often misunderstood as purely detective.

Detective Role

- Finding bugs during execution
- Reporting failures

Preventive Role

- Reviewing requirements
- Identifying ambiguity early
- Suggesting improvements
- Preventing defects before coding starts

Mature testing shifts effort **left** in the lifecycle.

2.6 Cost of Poor Quality (COPQ)

Defects cost more as they travel downstream.

Stage	Cost Impact
Requirements	Very Low
Design	Low
Development	Medium
Testing	High
Production	Very High

Early testing directly saves money.

2.7 Testing Does Not Eliminate Risk Completely

Testing:

- Reduces uncertainty
- Improves predictability

Testing does **not**:

- Guarantee zero defects
- Replace good development practices

Risk acceptance is a business decision, informed by testing results.

2.8 Summary of Section 2

- Testing exists to evaluate quality and reduce risk
- Defect detection is only one objective
- Testing supports both technical and business goals
- Early testing prevents costly failures
- Testing enables informed release decisions

Section 3: Errors, Defects, and Failures

3.1 Introduction

In software testing, **errors, defects, and failures** are not interchangeable terms.
They represent **different stages of a problem's life**.

Understanding their relationship is critical for:

- Root cause analysis
- Defect prevention
- Interview clarity

3.2 Error (Mistake)

Definition
An **error** is a human action that produces an incorrect result.

Errors occur due to:

- Misunderstanding requirements
- Logical mistakes
- Incorrect assumptions
- Lack of domain knowledge
- Time pressure or fatigue

Errors originate **in the mind**, not in the system.

Example
A developer misunderstands a requirement and writes incorrect logic.
That misunderstanding is the **error**.

3.3 Defect (Bug)

Definition
A **defect** is a flaw in the software where the actual behavior differs from the expected behavior.

Defects are:

- The **result of errors**
- Located in code, configuration, or documentation
- Detectable through testing or reviews

Example
Incorrect calculation logic implemented in the code is a **defect**.

3.4 Failure

Definition
A **failure** occurs when the software does not perform its intended function during execution.

Failures are:

- Visible to users
- Triggered when defects are executed
- The final outcome of unresolved defects

Example
Application crashes or produces wrong output during use.

3.5 Relationship Between Error, Defect, and Failure

The sequence is:

Error → Defect → Failure

- Human makes an error
- Error introduces a defect into the system
- Defect causes a failure when executed

However, not all defects result in failures.

3.6 Can a Defect Exist Without a Failure?

Yes.

Reasons:

- Defective code path is not executed
- Specific conditions not met
- Feature not yet exposed to users

This is why:

Absence of failures does not mean absence of defects.

3.7 Can a Failure Occur Without a Defect?

Yes, in rare cases.

Examples:

- Hardware failure
- Network outage
- Configuration mismatch
- Environmental issues

The system fails even though the software logic is correct.

3.8 Real-World Example (Interview-Oriented)

A banking application calculates interest incorrectly.

- Requirement misunderstood → **Error**
- Wrong formula coded → **Defect**
- Incorrect interest shown to customer → **Failure**

This explanation format is highly effective in interviews.

3.9 Why Testers Must Understand This Distinction

- Helps identify root cause
- Improves defect reporting quality
- Strengthens communication with developers
- Prevents blame-based discussions

Testing is about **facts and traceability**, not fault-finding.

3.10 Summary of Section 3

- Errors are human mistakes
- Defects are flaws in software
- Failures are visible incorrect behavior
- All failures come from defects, but not all defects cause failures
- Understanding this flow is fundamental to testing

Section 4: Verification vs Validation

4.1 Why Verification and Validation Matter

Building software fast is easy.
Building the **right software, correctly** is hard.

Verification and validation answer two different questions:

- Are we building the product **right**?
- Are we building the **right product**?

Confusing these two leads to perfectly built failures.

4.2 Verification

Definition
Verification is the process of evaluating work products to determine whether they meet specified requirements.

It focuses on:

- Correctness of design

- Compliance with specifications
- Early defect prevention

Verification answers:

“Are we building the product right?”

4.3 Characteristics of Verification

- Static activity (no code execution)
- Done early in SDLC
- Focuses on documents and designs
- Prevents defects

Common Verification Activities

- Requirement reviews
- Design reviews
- Code walkthroughs
- Inspections

Verification is about **thinking before building**.

4.4 Validation

Definition
Validation is the process of evaluating the software during execution to determine whether it satisfies user needs.

It focuses on:

- Actual behavior of the system
- User expectations
- Functional correctness

Validation answers:

“Are we building the right product?”

4.5 Characteristics of Validation

- Dynamic activity (code execution required)
- Done after development

- Focuses on the working product
- Detects defects

Common Validation Activities

- Functional testing
- System testing
- UAT
- Acceptance testing

Validation is about **experiencing the product**.

4.6 Key Differences Between Verification and Validation

Aspect	Verification	Validation
Objective	Build product correctly	Build correct product
Type	Static	Dynamic
Timing	Early SDLC	After development
Focus	Documents & design	Executable software
Performed by	Dev, QA, BA	QA, Users

4.7 Real-Time Example

Requirement
System should allow users to reset password using registered email.

- Checking requirement clarity → **Verification**
- Reviewing design flow → **Verification**
- Testing reset email functionality → **Validation**
- Confirming user can log in → **Validation**

4.8 Common Interview Traps

- Verification is not testing only
- Validation is not UAT only
- Both are required for quality
- Validation cannot fix wrong requirements

4.9 Why Both Are Required

- Verification prevents defects

- Validation detects defects
- One without the other causes failure

Together, they form the backbone of quality assurance.

4.10 Summary of Section 4

- Verification ensures correctness of build process
- Validation ensures fitness for use
- Verification is static, validation is dynamic
- Both are essential for successful software

Section 5: Quality Assurance (QA) vs Quality Control (QC)

5.1 Understanding "Quality" in Software

Quality in software is not accidental.
It is the result of **planned processes** and **controlled execution**.

QA and QC together ensure that:

- The right processes are followed
- The product meets defined standards

But they serve **different purposes**.

5.2 Quality Assurance (QA)

Definition
Quality Assurance is a process-focused activity that ensures the development process is designed and followed correctly to produce a quality product.

QA answers:

"Are we following the right process?"

5.3 Characteristics of QA

- Process-oriented
- Preventive in nature

- Applied throughout SDLC
- Focuses on improving processes
- Owned by the organization

QA Activities Include

- Defining standards
- Process audits
- Methodology selection
- Training and guidelines
- Continuous process improvement

QA ensures quality is **built-in**, not inspected later.

5.4 Quality Control (QC)

Definition
Quality Control is a product-focused activity that verifies whether the developed software meets the specified requirements.

QC answers:

"Does the product meet the requirements?"

5.5 Characteristics of QC

- Product-oriented
- Detective in nature
- Executed after development
- Focuses on defect detection
- Owned by the project team

QC Activities Include

- Testing
- Inspection of deliverables
- Defect reporting and tracking
- Validation activities

QC ensures quality is **verified**.

5.6 Key Differences Between QA and QC

Aspect	QA	QC
Focus	Process	Product
Nature	Preventive	Detective
Timing	Entire SDLC	After build
Responsibility	Organization-wide	Project-level
Example	Process audit	Test execution

5.7 Where Does Testing Fit?

Testing is primarily a **QC activity** because:

- It evaluates the product
- It identifies defects

However, testers also contribute to QA when they:

- Review requirements
- Suggest process improvements
- Participate in retrospectives

So:

Testing belongs to QC, but testers contribute to QA.

5.8 Relationship Between QA, QC, and Testing

- QA defines **how** to build
- QC checks **what** was built
- Testing verifies **if** it works as expected

They are complementary, not competing.

5.9 Interview Perspective

Interviewers look for:

- Clear distinction
- No blame language
- Process awareness

Avoid saying:

- QA = testing (incorrect)
- QC = debugging (incorrect)

5.10 Summary of Section 5

- QA focuses on process prevention
- QC focuses on product detection
- Testing is part of QC
- Both are essential for quality delivery

Section 6: Severity vs Priority

6.1 Why Severity and Priority Matter

Not all defects are equal.

Some defects:

- Break the system
- Annoy users
- Block releases
- Can wait

Severity and priority help teams **decide what to fix first**.

6.2 Severity

Definition
Severity indicates the **impact of a defect on the system's functionality**.

Severity answers:

"How badly does this defect affect the system?"

6.3 Severity Levels (Common Classification)

1. **Critical / Blocker**
    - System crash
    - Data loss

- No workaround

2. **High**
   - Major feature not working
   - Core business flow impacted
3. **Medium**
   - Partial functionality failure
   - Workaround exists
4. **Low**
   - Minor issue
   - Cosmetic defects

Severity is usually **decided by testers**.

6.4 Priority

Definition
Priority indicates **how soon a defect must be fixed**.

Priority answers:

"How urgently does this defect need to be resolved?"

6.5 Priority Levels (Common Classification)

1. **High**
   - Must be fixed immediately
   - Blocks release
2. **Medium**
   - Should be fixed in upcoming build
3. **Low**
   - Can be fixed later

Priority is usually **decided by business, product owner, or project manager**.

6.6 Key Differences Between Severity and Priority

Aspect	Severity	Priority
Meaning	Impact on system	Urgency of fix
Focus	Technical	Business
Decided by	Tester	Business/PM
Changes?	Rare	Frequent

6.7 Common Severity–Priority Combinations (Interview Gold)

High Severity – Low Priority

- Rarely used admin feature crashes
- Severe impact but low business urgency

Low Severity – High Priority

- Spelling mistake on homepage
- Minor issue but high visibility

6.8 Real-Time Explanation Pattern

When explaining in interviews:

1. State definition
2. Explain who decides
3. Give one combination example

This shows clarity and experience.

6.9 Why Misclassification Is Risky

- Wrong focus leads to wasted effort
- Critical issues may be delayed
- Business dissatisfaction increases

Severity and priority alignment keeps delivery efficient.

6.10 Summary of Section 6

- Severity measures impact
- Priority measures urgency
- Testers set severity
- Business sets priority
- Both guide defect resolution strategy

Section 7: Principles of Software Testing

7.1 Why Testing Principles Exist

Testing principles are **guiding rules**, not rigid laws.
They exist because decades of projects failed in predictable ways.

Understanding principles helps testers:

- Avoid unrealistic expectations
- Design effective tests
- Communicate testing limits clearly

7.2 Principle 1: Testing Shows Presence of Defects

Statement
Testing can show that defects are present, but cannot prove that defects are absent.

Explanation

- Passing tests does not mean defect-free software
- It only means tested scenarios worked

Interview Insight
Testing reduces uncertainty, not eliminates it.

7.3 Principle 2: Exhaustive Testing Is Impossible

Statement
It is impossible to test all combinations of inputs and conditions.

Explanation

- Time and cost constraints
- Infinite input combinations
- Complex integrations

Result
Testers apply:

- Risk-based testing
- Test design techniques

7.4 Principle 3: Early Testing

Statement
Testing activities should start as early as possible.

Explanation

- Defects found early are cheaper to fix
- Requirement reviews prevent downstream failures

Early testing is **defect prevention**.

7.5 Principle 4: Defect Clustering

Statement
A small number of modules contain most defects.

Explanation

- Complex or frequently changed modules
- Poorly designed components

Application
Focus testing effort where defects cluster.

7.6 Principle 5: Pesticide Paradox

Statement
Repeating the same tests eventually stops finding new defects.

Explanation

- Software adapts
- Test cases become stale

Solution

- Review test cases
- Add new scenarios
- Improve coverage

7.7 Principle 6: Testing Is Context Dependent

Statement
Testing approach depends on the application context.

Explanation

- Banking apps focus on security and accuracy
- Gaming apps focus on performance and usability

No one-size-fits-all testing.

7.8 Principle 7: Absence-of-Errors Fallacy

Statement
Even defect-free software is useless if it does not meet user needs.

Explanation

- Correctly built wrong product still fails

Validation is as important as verification.

7.9 Interview Perspective

Interviewers expect:

- Principle explanation
- One-line example
- No memorization tone

Understanding > reciting.

7.10 Summary of Section 7

- Principles guide testing strategy
- Testing proves presence, not absence
- Early testing saves cost
- Context defines approach

Section 8: Entry & Exit Criteria

8.1 Why Entry and Exit Criteria Are Needed

Testing without boundaries leads to:

- Chaos
- Missed expectations
- Endless cycles

Entry and exit criteria define **when testing starts and when it can stop**.

They act as **quality checkpoints**.

8.2 Entry Criteria

Definition
Entry criteria are the predefined conditions that must be met before testing activities can begin.

Entry criteria answer:

“Is the system ready to be tested?”

8.3 Common Entry Criteria Examples

- Approved and stable requirements
- Test cases prepared and reviewed
- Test environment ready
- Build deployed successfully
- Required tools and data available

Entry criteria protect testers from testing unstable builds.

8.4 Exit Criteria

Definition
Exit criteria are the conditions that must be satisfied to conclude testing.

Exit criteria answer:

“Is the testing sufficient to stop?”

8.5 Common Exit Criteria Examples

- All planned test cases executed
- Critical and high defects fixed and closed

- Acceptable defect density achieved
- Test coverage met
- Stakeholder sign-off

Exit criteria ensure **controlled release decisions**.

8.6 Who Defines Entry and Exit Criteria?

- Test Lead or Test Manager
- QA team
- Project stakeholders

They are usually documented in:

- Test Plan
- Test Strategy

8.7 What If Exit Criteria Is Not Met?

Options include:

- Extend testing
- Defer low-risk defects
- Accept risk with approval
- Delay release

Testing informs decisions, it does not make them.

8.8 Interview Trap Question

Can testing stop if exit criteria is not met?

Correct answer:

Yes, with business approval and documented risk acceptance.

8.9 Importance of Entry & Exit Criteria

- Prevent premature testing
- Avoid endless testing
- Improve transparency

- Enable objective decisions

8.10 Summary of Section 8

- Entry criteria define readiness
- Exit criteria define completeness
- Both are planned in advance
- They support quality governance

Section 9: Role & Responsibilities of a Tester

9.1 Tester's Role in a Software Project

A tester's role is **not limited to executing test cases**.
A tester is a **quality advocate** who represents the end user while understanding business constraints.

At a high level, a tester:

- Evaluates product quality
- Identifies risks early
- Provides objective feedback
- Supports informed release decisions

9.2 Tester's Role Across SDLC

- **Requirements Phase**
  - Review requirements for clarity, completeness, testability
  - Identify ambiguities and gaps
- **Design Phase**
  - Review design documents
  - Identify risk areas and dependencies
- **Development Phase**
  - Prepare test cases and test data
  - Participate in walkthroughs
- **Testing Phase**
  - Execute test cases
  - Report and track defects
  - Perform retesting and regression
- **Release Phase**
  - Provide test summary and quality status

- Support go/no-go decisions

9.3 Tester's Responsibilities

- Understanding requirements
- Designing effective test scenarios
- Executing tests systematically
- Logging defects with clarity
- Collaborating with developers and business
- Maintaining test artifacts
- Ensuring coverage and traceability

A tester owns **quality insight**, not just defects.

9.4 Responsibility vs Authority

Testers:

- Are responsible for reporting quality status
- Are not solely responsible for final release decisions

Quality is a **shared responsibility**.

9.5 Summary of Section 9

- Tester is a quality representative
- Role spans entire lifecycle
- Focus is on risk, not blame
- Responsibility is insight, not decision-making

Section 10: Qualities of a Good Tester

10.1 Why Tester Qualities Matter

Tools can be taught.
Mindset cannot.

Good testers are defined by **how they think**, not just what they know.

10.2 Technical Qualities

- Understanding of SDLC and STLC
- Knowledge of testing techniques
- Awareness of application architecture
- Basic understanding of databases and web concepts

10.3 Analytical Qualities

- Logical thinking
- Attention to detail
- Ability to break systems into components
- Risk identification

Testers think in **what-if patterns**.

10.4 Behavioral & Professional Qualities

- Curiosity
- Patience
- Persistence
- Objectivity
- Accountability

Good testers challenge politely, not emotionally.

10.5 Communication Skills

- Clear defect reporting
- Precise explanation of issues
- Effective collaboration with teams
- Ability to explain technical issues to non-technical stakeholders

Communication turns defects into fixes.

10.6 Business Mindset

- Understanding user impact
- Prioritizing based on risk

- Thinking beyond test cases

Interviewers silently look for this mindset.

10.7 Summary of Section 10

- Good testers combine skill and mindset
- Curiosity and clarity are critical
- Communication amplifies testing value

Section 11: Testing vs Debugging

11.1 Definition of Testing

Testing is the process of evaluating software to identify defects and assess quality.

Testing answers:

“Does the software behave as expected?”

11.2 Definition of Debugging

Debugging is the process of locating, analyzing, and fixing defects in the code.

Debugging answers:

“Why is the software behaving incorrectly?”

11.3 Key Differences Between Testing and Debugging

Aspect	Testing	Debugging
Objective	Find defects	Fix defects
Focus	Product behavior	Code logic
Performed by	Testers	Developers
Nature	Quality assessment	Problem resolution

11.4 Relationship Between Testing and Debugging

- Testing identifies the problem
- Debugging resolves the problem
- Both are dependent but distinct activities

Testers may assist debugging by:

- Providing clear reproduction steps
- Supplying logs and evidence

But fixing is not the tester's responsibility.

11.5 Interview Trap

Incorrect statement:

Testers debug defects.

Correct framing:

Testers help diagnose issues, but debugging is owned by developers.

11.6 Summary of Section 11

- Testing finds problems
- Debugging fixes problems
- Both are essential but separate
- Clear ownership prevents conflict

QUESTIONS TO CHECK YOUR LEARNINGS

Core Definition & Purpose

1. What is software testing?
2. Why is software testing required?
3. What are the objectives of software testing?
4. Can software testing ensure 100% quality? Why or why not?
5. What happens if testing is skipped in a project?

Errors, Defects & Failures

6. What is an error?
7. What is a defect (bug)?
8. What is a failure?
9. Explain the difference between error, defect, and failure.
10. Can a defect exist without a failure? Explain with an example.

Verification vs Validation

11. What is verification?
12. What is validation?
13. Explain verification vs validation with a real-time example.
14. Is verification done before or after validation?
15. Who performs verification and validation?

QA vs QC

16. What is Quality Assurance?
17. What is Quality Control?
18. Difference between QA and QC.
19. Is testing part of QA or QC?
20. Can QA exist without QC?

Severity & Priority

21. What is severity?
22. What is priority?
23. Difference between severity and priority.
24. Give examples of:

- High severity & low priority
- Low severity & high priority

25. Who decides severity and priority?

Testing Principles (Very Important)

26. What are the principles of software testing?
27. Explain “Testing shows presence of defects”.
28. What is exhaustive testing?
29. What is defect clustering?
30. Explain the pesticide paradox.
31. What is early testing?
32. What is context-dependent testing?

When & Why Testing

33. When should testing start in SDLC?
34. Why early testing is important?
35. What is the cost of defect fixing at different stages?
36. What are the risks of late testing?

Entry & Exit Criteria

37. What is entry criteria in testing?
38. What is exit criteria?
39. Who defines entry and exit criteria?
40. Can testing stop if exit criteria is not met?

Roles & Responsibilities

41. What is the role of a tester in a project?
42. What qualities should a good tester have?
43. Difference between developer testing and tester testing.
44. Can testing be done by developers alone? Why?

Misconceptions & Tricky Questions

45. Is testing only about finding bugs?
46. Is testing a responsibility of testers only?
47. Can a bug-free application exist?
48. Why is testing considered a preventive activity?
49. What is the difference between testing and debugging?
50. What do you understand by quality in software testing?

Chapter 2: SDLC & STLC

CLUSTER 1: SDLC – Concept, Purpose & Big Picture

1️⃣ What is SDLC? (Not textbook, interview-safe)

SDLC (Software Development Life Cycle) is a structured framework that defines **how software is planned, built, tested, deployed, and maintained** in a controlled and predictable manner.

👉 Interview insight:
SDLC is not about coding speed. It is about **risk reduction, quality assurance, and predictable delivery**.

2️⃣ Why SDLC exists (This is often asked indirectly)

Without SDLC:

- Requirements change uncontrollably
- Defects are found late (costly)
- Delivery timelines slip
- Accountability becomes unclear

With SDLC:

- Clear phase ownership
- Early defect detection
- Planned testing
- Controlled releases

💡 One-liner you can use:

SDLC helps organizations deliver quality software within time and cost by reducing uncertainty and rework.

3️⃣ Core Phases of SDLC (High-level first)

Every SDLC model contains these **logical phases**, even if names differ:

1. Requirement Gathering & Analysis

2. Design
3. Development (Coding)
4. Testing
5. Deployment
6. Maintenance

📌 Important:
Testing is **not one phase**, it is a **continuous activity** across SDLC.

4️⃣ Where does Testing fit in SDLC? (Very important)

❌ Wrong thinking:

Testing starts after development

✅ Correct interview answer:

Testing activities begin from the requirement phase itself and continue till maintenance.

Testing presence in each phase:

- Requirement → Requirement review, ambiguity identification
- Design → Testability review, scenario identification
- Development → Unit & integration validation support
- Testing → Execution, defect reporting
- Deployment → Smoke & sanity
- Maintenance → Regression testing

This shows **maturity**, not junior thinking.

5️⃣ SDLC vs Real Projects (Ground reality line)

You can say:

While SDLC models are defined theoretically, in real projects organizations tailor them based on timelines, risk, and client needs.

This signals experience.

6️⃣ Common Interview Trap (Avoid this)

❌ Saying:

We follow Agile, so SDLC is not there

✅ Correct framing:

Agile is also an SDLC model, but it follows an iterative and incremental approach.

7️⃣ Mini Summary (Use this to close answers)

SDLC provides a structured flow for software delivery, while testing plays a continuous role across phases to ensure quality, reduce risk, and support predictable releases.

Great. Let's build this layer by layer, like a solid test strategy, not a rushed execution 😌

CLUSTER 2: SDLC Models (Waterfall, V-Model, Agile) – Interview-Focused

---

1️⃣ Why SDLC Models exist (Concept clarity)

An **SDLC model** defines:

- **Order of phases**
- **Flow of activities**
- **When testing happens**
- **How changes are handled**

💡 Interview-grade line:

SDLC models provide a predictable structure for execution, coordination, and quality control.

2️⃣ Waterfall Model (Traditional & foundational)

What it is

A **sequential model** where each phase starts **only after the previous phase is completed**.

Flow
Requirements → Design → Development → Testing → Deployment → Maintenance

---

Key Characteristics

- No overlapping phases
- Heavy documentation
- Changes are difficult
- Testing starts late

Advantages

- Simple and easy to manage
- Clear milestones
- Suitable for stable requirements

Disadvantages (Very important)

- Late defect detection
- High cost of change
- Not suitable for evolving requirements

Interview insight

❗ Never say “Waterfall is outdated”.

Say:

Waterfall works well when requirements are fixed and clearly defined, such as regulatory or legacy systems.

3️⃣ V-Model (Verification & Validation Model)

What it is

An extension of Waterfall where **each development phase has a corresponding testing phase**.

Left side (Verification)
Requirements → Design → Coding

Right side (Validation)
Acceptance → System → Integration → Unit Testing

Key Concept (Very important)

Test planning and design start in parallel with development phases.

Mapping Example (You MUST remember this)

- Business Requirements → UAT
- System Requirements → System Testing
- High-Level Design → Integration Testing
- Low-Level Design → Unit Testing

Advantages

- Early testing
- Clear traceability
- Reduced defect leakage

Limitations

- Rigid like Waterfall
- Not suitable for frequent changes

Interview killer line:

V-Model emphasizes early test design and traceability, reducing late-stage defects.

4️⃣ Agile Model (Modern & mandatory)

What Agile is (Correct framing)

Agile is an **iterative and incremental SDLC model** focused on:

- Customer collaboration
- Continuous feedback
- Frequent releases

Agile Key Terms (Interview radar 🚨)

- Sprint
- User Story
- Backlog
- Increment
- Definition of Done

How testing works in Agile

- Testing starts on Day 1
- Testers work in the same sprint as developers
- Continuous testing
- Regression every sprint

Agile Testing Mindset

❌ Separate testing phase
✅ Testing is part of development

Advantages

- Early feedback
- Faster defect detection
- High adaptability

Challenges (Be honest but mature)

- Less documentation
- Tight timelines
- Requires skilled testers

Agile interview-ready line:

In Agile, testing is continuous and collaborative, ensuring quality at every sprint increment.

5️⃣ Comparing Models (Very Common Question)

Aspect	Waterfall	V-Model	Agile
Requirement change	Difficult	Difficult	Easy
Testing start	Late	Early	Continuous
Documentation	Heavy	Heavy	Light
Delivery	Single	Single	Incremental

6️⃣ Model selection (Scenario-based question)

You can say:

The choice of SDLC model depends on requirement stability, risk, timeline, and customer involvement.

7️⃣ Mini Summary (Close strong)

Waterfall and V-Model focus on predictability and documentation, while Agile focuses on adaptability and continuous testing.

Nice. You're progressing exactly how a **strong interviewer mind** progresses.
Now we switch from **development lens → tester lens** 🎯

CLUSTER 3: STLC – Purpose, Flow & Relation with SDLC

1️⃣ What is STLC? (Interview-grade definition)

STLC (Software Testing Life Cycle) is a systematic process that defines **testing-specific activities**, their sequence, and deliverables to ensure software quality.

💡 One-liner you can use:

STLC defines *how* testing is planned, designed, executed, and closed in a structured manner.

2️⃣ Why STLC is needed (Very common "why" question)

Testing without STLC leads to:

- Missed coverage
- Ad-hoc execution
- Poor defect tracking
- No closure metrics

STLC ensures:

- Planned testing
- Measurable quality
- Traceability
- Controlled execution

📌 Interview maturity line:

STLC brings discipline, predictability, and accountability to testing activities.

3️⃣ STLC vs SDLC (Clear separation is important)

SDLC	STLC
Product-focused	Quality-focused
Covers end-to-end software	Covers testing activities
Owned by entire team	Owned mainly by QA
Broader lifecycle	Subset of SDLC

👉 Interview-safe conclusion:

STLC operates within SDLC and aligns testing activities with development phases.

4️⃣ Phases of STLC (High-level first)

Every project, Agile or not, follows these **logical phases**:

1. Requirement Analysis
2. Test Planning

3. Test Case Design
4. Test Environment Setup
5. Test Execution
6. Defect Reporting & Tracking
7. Test Closure

📌 Names may change, intent does not.

5️⃣ Entry & Exit Criteria (Often missed, very important)

Each STLC phase has:

- **Entry Criteria**: Conditions to start
- **Exit Criteria**: Conditions to stop

Example:

- Entry: Approved requirements
- Exit: Test cases reviewed & approved

💡 Strong line:

Entry and exit criteria ensure controlled progression and prevent premature execution.

6️⃣ STLC in Agile vs Traditional (Trap question)

❌ Wrong:

Agile doesn't follow STLC

✅ Correct:

Agile follows STLC, but in a compressed and iterative manner.

How:

- Requirement analysis → user stories
- Planning → sprint planning
- Design → ongoing
- Execution → within sprint
- Closure → sprint review/retro

7️⃣ Tester's responsibility in STLC

Testers are responsible for:

- Understanding requirements
- Designing test cases
- Executing tests
- Reporting defects
- Providing quality metrics
- Giving release confidence

💬 Interview framing:

STLC ensures testers contribute not just by finding defects, but by ensuring release readiness.

8️⃣ Mini Summary (Close answers confidently)

STLC provides a structured testing flow aligned with SDLC, ensuring quality is planned, executed, measured, and formally closed.

Excellent. Now we enter the **most underrated but most powerful STLC phase**.
Interviewers judge seniority *right here* 👀

CLUSTER 4: STLC Phase 1 – Requirement Analysis (Deep & Practical)

1️⃣ Purpose of Requirement Analysis (Tester's perspective)

Requirement Analysis in STLC is **not just reading documents**.

It is about:

- Understanding what needs to be tested
- Identifying ambiguities early
- Ensuring testability
- Preventing defects before coding starts

🎯 Interview-ready line:

Requirement analysis helps testers prevent defects rather than only detect them.

2️⃣ Inputs for Requirement Analysis

Testers typically analyze:

- BRD (Business Requirement Document)
- SRS / FRS
- User Stories & Acceptance Criteria (Agile)
- Wireframes / Mockups
- API contracts (if applicable)

📌 Key insight:
Requirements are **not always complete or perfect**.

3️⃣ Tester Activities in Requirement Analysis

During this phase, testers:

- Review requirements for clarity
- Identify missing scenarios
- Ask clarifying questions
- Identify testable vs non-testable requirements
- Start high-level test scenarios
- Identify dependencies & risks

💬 Interview phrase:

Testers actively participate in requirement reviews to reduce ambiguity and rework.

4️⃣ Common Requirement Issues Testers Find (Very realistic)

- Missing validation rules
- Unclear error messages
- Boundary conditions not specified
- Negative scenarios not mentioned
- Non-functional requirements missing

You can say:

Many defects originate from unclear requirements rather than faulty code.

This sounds experienced.

5️⃣ What is RTM? (Traceability Matrix)

RTM (Requirement Traceability Matrix) maps:

- Requirements ↔ Test Scenarios / Test Cases ↔ Defects

Purpose:

- Ensure 100% requirement coverage
- Track impact of changes
- Avoid missed testing

6️⃣ Types of Traceability

- Forward Traceability
  Requirement → Test Case
- Backward Traceability
  Test Case → Requirement

🎯 Strong interview line:

RTM ensures that every requirement is tested and no test is written without a purpose.

7️⃣ Entry & Exit Criteria – Requirement Analysis

Entry Criteria

- Approved requirements
- Access to requirement documents
- Stakeholder availability

Exit Criteria

- Requirements reviewed
- Queries clarified
- Test scenarios identified
- RTM draft prepared

8️⃣ Challenges in Requirement Analysis (Be honest)

- Frequent requirement changes
- Ambiguous user stories
- Time constraints
- Stakeholder unavailability

Interview-safe framing:

These challenges are mitigated through continuous communication and early clarification.

9️⃣ Requirement Analysis in Agile (Must know)

In Agile:

- Requirements come as user stories
- Acceptance criteria guide testing
- Refinement sessions help testers prepare early

📌 Good line:

In Agile, requirement analysis is continuous and happens sprint by sprint.

🔟 Mini Summary (Close strong)

Requirement analysis is the foundation of effective testing, enabling early defect prevention, better coverage, and reduced rework.

📘 Business Requirements Brief (BRB)

1. Document Overview

Item	Description
Document Name	Business Requirements Brief – Customer Login & Access
Version	1.0
Prepared By	Business Team
Date	15-Jan-2026
Application	Customer Web Portal
Target Users	Registered Customers
Platforms	Web (Desktop & Mobile)

---

2. Business Objective

The objective of this requirement is to provide **secure and controlled access** to registered customers so they can view and manage their account information while ensuring compliance with security and audit standards.

---

3. In-Scope

ID	In-Scope Items
IS-01	Customer login using email and password
IS-02	Validation of login credentials
IS-03	Account lock after consecutive failed attempts
IS-04	Forgot password functionality
IS-05	Audit logging for login activities

---

4. Out-of-Scope

ID	Out-of-Scope Items
OS-01	Social login (Google, Facebook, etc.)
OS-02	Two-factor authentication
OS-03	Admin user login

---

5. Business Rules (Very Important for Testing)

Rule ID	Business Rule
BR-01	User must provide a registered email ID to log in
BR-02	Password must meet system-defined security rules
BR-03	Account must be locked after **5 consecutive failed login attempts**
BR-04	Locked accounts must be unlocked automatically after **30 minutes**
BR-05	Password reset link must expire after **15 minutes**
BR-06	Error messages must not expose system or security details

---

6. Functional Requirements

FR ID	Requirement Description
FR-01	System shall allow login using valid email and password
FR-02	System shall display error message for invalid credentials
FR-03	System shall lock account after 5 consecutive failures
FR-04	System shall display locked account message
FR-05	System shall allow users to reset password via email
FR-06	System shall log all login attempts for audit

---

7. Non-Functional Requirements

NFR ID	Requirement
NFR-01	Login page shall load within 3 seconds
NFR-02	System shall support Chrome, Edge, Firefox
NFR-03	System shall support concurrent logins up to 5,000 users
NFR-04	Login data must be encrypted in transit

---

8. User Flow (High Level)

Step	Description
1	User navigates to login page
2	User enters email and password
3	System validates credentials
4	System grants or denies access
5	User is redirected or shown error

---

9. Assumptions & Dependencies

ID	Description
AD-01	User is already registered
AD-02	Email service is available
AD-03	User has internet connectivity

---

10. Acceptance Criteria

AC ID	Criteria
AC-01	User can log in with valid credentials
AC-02	Invalid login shows error message
AC-03	Account locks after 5 failures
AC-04	Password reset works as expected
AC-05	Audit logs are generated

🧠 How YOU Should Analyze This BRB (Interview Perspective)

From this BRB, you should be able to:

Derive:

- Test Scenarios from **FRs + BRs**
- Test Cases from **each business rule**
- RTM using **FR ID ↔ Test Case ID**
- Edge cases using **Assumptions**

Example RTM Snippet (Your practice)

FR ID	Business Rule	Test Case ID
FR-03	BR-03	TC-LOGIN-010
FR-04	BR-04	TC-LOGIN-011
FR-05	BR-05	TC-LOGIN-012

Interview-Ready Line You Can Say 💼

I analyze BRBs by identifying business rules, functional and non-functional requirements, and then map them to test scenarios and RTMs to ensure complete coverage and traceability.

📄 **Sample Business Requirement (Jira – Wall of Words Style)**

Epic: Customer Login & Account Access Management

Business Context:
The application provides registered customers access to their account dashboard, transaction history, and profile management features. Customers must be able to log in securely using their registered credentials. The login functionality should support both desktop and mobile users and comply with security and compliance requirements.

Business Requirement Description (As it appears in Jira)

Users should be able to log in to the application using their registered email address and password. The system should validate the entered credentials and grant access to the user dashboard upon successful authentication. If the user enters invalid credentials, the system should display an appropriate error message and should not allow access to the dashboard.

The system should restrict access after multiple failed login attempts to prevent unauthorized access. Currently, the business expects the system to lock the account temporarily if invalid credentials are entered multiple times consecutively. The exact number of attempts may be finalized later, but the expectation is that security should not be compromised.

The login page should be responsive and should work consistently across supported browsers and devices. The page load time should be reasonable, and the system should handle concurrent login requests without performance degradation.

Users who have forgotten their password should be able to reset it using the “Forgot Password” option. The password reset process should be secure and should ensure that only the registered user is able to reset the password. The password reset link should expire after a certain duration for security reasons.

Error messages displayed during login should be user-friendly and should not expose sensitive system or security information. However, the system should clearly indicate whether the login failure is due to invalid credentials or account lock.

The application should maintain audit logs for login attempts for compliance and monitoring purposes. These logs should be accessible to authorized internal users only.

Acceptance Criteria (Partially Defined, Typical Jira)

- User must be able to log in with valid credentials
- Invalid credentials should not allow login
- Account should be locked after multiple failed attempts
- Forgot password flow should work
- Login should be secure

(Yes, this is intentionally vague 😄)

🎯 **Your Task as a Tester (What interviewers expect)**

From this **single wall-of-words requirement**, you can derive **RTMs at different levels**.

Below I show you **how to structure them**, but I will NOT fill everything. You should practice.

🔹 **Level 1 RTM – High-Level Business RTM**

Purpose: Ensure every business requirement is covered.

BR ID	Business Requirement	Test Scenario ID	Coverage
BR-01	User login with valid credentials	TS-LOGIN-01	Covered
BR-02	Error handling for invalid login	TS-LOGIN-02	Covered
BR-03	Account lock after failures	TS-LOGIN-03	Covered
BR-04	Forgot password functionality	TS-LOGIN-04	Covered

BR ID	Business Requirement	Test Scenario ID	Coverage
BR-05	Security & audit logging	TS-LOGIN-05	Covered

👉 This level is often used for **status reporting**.

🔹 Level 2 RTM – Functional RTM (Requirement → Test Case)

Purpose: Detailed functional coverage.

Req ID	Requirement Description	Test Case ID	Status
FR-01	Login with valid email & password	TC-LOGIN-001	Not Run
FR-02	Invalid password error message	TC-LOGIN-002	Not Run
FR-03	Invalid email error handling	TC-LOGIN-003	Not Run
FR-04	Account lock after X failures	TC-LOGIN-004	Not Run
FR-05	Locked account message display	TC-LOGIN-005	Not Run

🔹 Level 3 RTM – Requirement → Test Case → Defect

Purpose: Used during execution & audits.

Req ID	Test Case ID	Defect ID	Defect Status
FR-02	TC-LOGIN-002	BUG-1012	Open
FR-04	TC-LOGIN-004	BUG-1018	Fixed
FR-05	TC-LOGIN-005	BUG-1021	Reopen

🔹 Level 4 RTM – Multi-Dimensional (Senior Level)

Purpose: Used by leads.

Req ID	Scenario	Test Case	Automation	Defect	Release
FR-01	TS-01	TC-001	Yes	–	R1
FR-02	TS-02	TC-002	No	BUG-1012	R1
FR-04	TS-03	TC-004	Yes	BUG-1018	R1

🧠 Hidden Ambiguities (Interview Gold)

When analyzing this requirement, you SHOULD ask:

- What is the exact number of failed attempts?
- Lock duration?
- Password complexity rules?
- Browser support list?
- Expiry time for reset link?
- Should error messages differ for email vs password?
- What defines “reasonable performance”?

Mentioning these in interviews shows **real analysis skill**.

What to do next

1. Create **test scenarios** from this
2. Write **10–15 test cases**
3. Prepare **RTM Level 2**
4. Identify **gaps & questions**

CLUSTER 5: STLC Phase 2 – Test Planning (Deep, Practical, Interview-Ready)

This cluster is **high signal**. Many testers *execute* tests; fewer understand *planning*. Interviewers notice.

1️⃣ What is Test Planning? (Correct framing)

Test Planning is the phase where the testing approach, scope, resources, schedule, risks, and deliverables are defined **before execution begins**.

🎯 Interview-grade one-liner:

Test planning defines how, when, and what to test to meet quality objectives.

2️⃣ Objectives of Test Planning

Test planning ensures:

- Clear testing scope
- Resource & effort estimation
- Risk identification
- Defined entry & exit criteria
- Alignment with project goals

💡 Senior-level line:

Effective test planning minimizes surprises during execution.

3️⃣ Inputs to Test Planning

Test planning typically uses:

- Approved requirements (BRB / SRS / User Stories)
- Project timelines
- Resource availability
- Test strategy or org standards
- Historical metrics (if available)

4️⃣ Key Deliverable: Test Plan

What is a Test Plan?

A **Test Plan** is a formal document describing:

- Scope
- Approach
- Resources
- Schedule
- Risks
- Deliverables

5️⃣ Contents of a Test Plan (Must Memorize)

Section	Purpose
Introduction	Overview & objectives
Scope	In-scope & out-of-scope
Test Strategy	Overall testing approach
Test Types	Functional, regression, etc.
Environment	Test setup details
Entry/Exit Criteria	Control points
Roles & Responsibilities	Ownership
Schedule	Timelines
Risks & Mitigation	Risk handling
Deliverables	Outputs
Metrics	Measurement

Section	Purpose
Approvals	Sign-off

📌 Interview tip:
Never say “test plan contains test cases”. It does NOT.

6️⃣ Test Strategy vs Test Plan (Very common question)

Aspect	Test Strategy	Test Plan
Scope	Organization-level	Project-level
Ownership	Management	Test Lead
Change frequency	Rare	Frequent
Focus	What & how	Execution details

🎯 One-liner:

Test strategy defines the vision; test plan defines execution.

7️⃣ Estimation in Test Planning

Test effort is estimated based on:

- Number of requirements
- Complexity
- Test types
- Resource skill level
- Past project data

Common techniques:

- Work Breakdown Structure (WBS)
- Expert judgment
- Analogous estimation

8️⃣ Risk Identification (Interview Favorite)

Examples of Testing Risks:

- Unstable requirements
- Limited test environment
- Tight deadlines

- Dependency on external teams

Mitigation Example:

Risk: Frequent requirement changes
Mitigation: Early review, flexible test design

9️⃣ Entry & Exit Criteria (Test Planning Phase)

Entry Criteria

- Requirements finalized
- Resources identified
- Project timeline available

Exit Criteria

- Test plan reviewed
- Test plan approved
- Risks documented

🔟 Test Planning in Agile (Important)

In Agile:

- Test planning happens during sprint planning
- Documentation is lightweight
- Focus is on collaboration

Good line:

Agile test planning is continuous and adapts sprint by sprint.

1️⃣1️⃣ Mini Summary (Close Strong)

Test planning lays the foundation for effective testing by defining scope, strategy, risks, and resources before execution begins.

CLUSTER 6: STLC Phase 3 – Test Scenarios & Test Case Design

1️⃣ Purpose of Test Case Design

Test case design converts **requirements into verifiable checks**.

🎯 Interview-grade line:

Test case design ensures requirements are validated systematically and repeatedly.

2️⃣ Test Scenario vs Test Case (Classic Question)

Test Scenario

- High-level testing condition
- Answers *what to test*
- Derived from requirements

Example:

Verify login functionality

Test Case

- Detailed steps to execute
- Answers *how to test*
- Includes input, steps, and expected result

Example:

Enter valid email and password and verify dashboard access

Comparison Table

Aspect	Test Scenario	Test Case
Detail level	High-level	Detailed
Count	Few	Many
Usage	Planning & coverage	Execution

🎯 One-liner:

Scenarios define coverage; test cases define execution.

3️⃣ Inputs for Test Design

Test cases are designed using:

- Approved requirements
- Business rules
- RTM
- User flows
- Design documents
- Past defect data

4️⃣ Components of a Test Case (Must Know)

Field	Purpose
Test Case ID	Unique identification
Description	What is being tested
Preconditions	Setup required
Test Steps	Actions
Test Data	Input values
Expected Result	Expected outcome
Actual Result	Observed outcome
Status	Pass/Fail
Remarks	Comments

5️⃣ Test Design Techniques (Interview Magnet)

1. Equivalence Partitioning

- Divide input data into valid & invalid classes
- Reduces number of test cases

Example:

- Password length 8–16 → valid
- <8 or >16 → invalid

---

2. Boundary Value Analysis

- Focus on edges of input range

Example:

- Min: 8, Max: 16
- Test: 7, 8, 9, 15, 16, 17

---

3. Decision Table Testing

- Used when multiple conditions affect outcome

Example:

- Login allowed based on account status & credentials

---

4. State Transition Testing

- System behavior depends on state

Example:

- Active → Locked → Unlocked account

---

5. Error Guessing

- Based on experience
- No formal rules

Interview tip:

Error guessing improves test coverage using past defects and domain knowledge.

6️ Positive & Negative Test Cases

- **Positive**: Valid inputs, expected flow
- **Negative**: Invalid inputs, edge cases

Good line:

Negative testing ensures system robustness against invalid usage.

7️ Test Coverage & Traceability

Coverage ensured by:

- Mapping test cases to requirements
- Maintaining RTM
- Reviewing test cases

8️⃣ Test Case Review (Often ignored)

Purpose:

- Identify missing scenarios
- Improve clarity
- Remove duplicates

Participants:

- Testers
- Leads
- Business analysts

9️⃣ Test Case Design in Agile

- Derived from user stories
- Acceptance criteria drive test cases
- Designed within sprint
- Lightweight documentation

🔟 Mini Summary

Test case design transforms requirements into executable validations using structured techniques to ensure complete coverage and quality.

CLUSTER 7: STLC Phase 4 – Test Environment Setup & Test Data

1️⃣ What is a Test Environment?

A **test environment** is the combination of:

- Hardware

- Software
- Network
- Application build
- Test data

used to execute test cases.

🎯 Interview-grade line:

A stable test environment ensures reliable and repeatable test execution.

2️⃣ Why Test Environment is Critical

If the environment is unstable:

- Defects are misleading
- Reproduction becomes difficult
- Testing timelines slip

You can say:

Many testing delays are environment-related rather than testing-related.

This shows maturity.

3️⃣ Components of a Test Environment

Component	Examples
Hardware	Servers, memory, CPU
Software	OS, browsers, DB
Application	Build/version
Network	VPN, firewall
Test Data	Valid/invalid data

4️⃣ Test Environment Setup Responsibility

Usually:

- DevOps / Infra → Environment creation
- QA → Environment validation

Interview-safe line:

QA validates environment readiness before execution.

5️⃣ Entry & Exit Criteria – Environment Setup

Entry Criteria

- Test cases ready
- Build available
- Environment access provided

Exit Criteria

- Environment validated
- Smoke test passed
- Test data prepared

6️⃣ What is Smoke Testing? (Environment Validation)

Smoke testing ensures:

- Application is stable
- Major features work
- Environment is usable

💡 One-liner:

Smoke testing confirms build stability before detailed testing.

7️⃣ Test Data Management (Often asked)

Test data includes:

- Valid user data
- Invalid data
- Boundary data
- Role-based data

Sources:

- Production-like data (masked)
- Dummy data
- Data created by QA

8️⃣ Challenges in Test Environment

Common issues:

- Environment not matching production
- Frequent build failures
- Data dependency
- Access issues

Interview framing:

Environment risks are mitigated through early coordination and smoke validation.

9️⃣ Test Environment in Agile

- Shared environments
- Frequent builds
- Quick smoke tests
- Rapid data refresh

🔟 Mini Summary

Test environment setup ensures the application and data are ready for reliable execution, reducing false defects and delays.

CLUSTER 8: STLC Phase 5 – Test Execution & Reporting

1️⃣ What is Test Execution?

Test Execution is the phase where:

- Test cases are executed
- Results are compared with expected outcomes
- Defects are identified and reported

🎯 Interview-grade line:

Test execution validates whether the application behaves as expected under defined conditions.

2️⃣ Preconditions for Test Execution

Before execution starts:

- Test environment is stable
- Build is deployed
- Test data is available
- Test cases are approved

💡 Strong line:

Execution should never start without environment validation.

3️⃣ Test Execution Activities

During execution, testers:

- Execute test cases
- Record actual results
- Mark pass/fail status
- Raise defects
- Re-test fixes
- Perform regression testing

4️⃣ Test Case Statuses (Must Know)

Status	Meaning
Pass	Expected result met
Fail	Expected result not met
Blocked	Cannot execute due to dependency
Not Run	Not executed yet

Interview tip:

Blocked tests indicate external issues, not test failure.

5️⃣ Defect Identification During Execution

A defect is raised when:

- Actual result ≠ Expected result
- Requirement is not met
- System behaves incorrectly

6️⃣ Retesting vs Regression (Classic)

Retesting	Regression
Tests fixed defects	Tests impacted areas
Limited scope	Broader scope
Specific	Reusable

🎯 One-liner:

Retesting confirms defect fix; regression ensures nothing else broke.

7️⃣ Test Reporting (Daily & Cycle)

Daily Status Report (DSR)

Includes:

- Executed test cases
- Passed/Failed count
- Defects raised
- Blockers

Test Execution Report

Includes:

- Overall coverage
- Defect summary
- Risks
- Release readiness

8️⃣ Handling Critical Defects (Scenario)

If a critical defect is found:

- Stop further execution (if needed)
- Inform stakeholders
- Log defect with high severity

- Participate in triage

Good line:

Critical defects require immediate visibility and triage.

9️⃣ Test Execution in Agile

- Tests executed within sprint
- Continuous testing
- Frequent regression
- Automation support

🔟 Mini Summary

Test execution validates application behavior, identifies defects, and provides real-time quality visibility through reporting.

CLUSTER 9: Defect Management, Defect Life Cycle & Triage

1️⃣ What is Defect Management?

Defect Management is the process of:

- Identifying
- Logging
- Tracking
- Verifying
- Closing defects

🎯 Interview-grade line:

Defect management ensures issues are systematically tracked until resolution.

2️⃣ What is a Defect? (Tester framing)

A defect is a deviation between:

- Expected result (requirement)

- Actual system behavior

You can also say:

A defect represents unmet requirements or incorrect system behavior.

3️⃣ Defect Life Cycle (Must Memorize)

Typical Defect States

1. New
2. Open
3. Assigned
4. Fixed
5. Retest
6. Closed
7. Reopen
8. Deferred / Rejected / Duplicate (optional)

📌 Interview insight:
Status names vary by tool, but **flow remains same**.

4️⃣ Defect Report Contents

Field	Purpose
Defect ID	Unique tracking
Summary	Short description
Steps to Reproduce	How to recreate
Expected Result	Intended behavior
Actual Result	Observed behavior
Severity	Impact
Priority	Urgency
Environment	Context
Attachments	Evidence

5️⃣ Severity vs Priority (Again, but deeper)

Severity	Priority
Technical impact	Business urgency
Set by QA	Set by Product

Severity	Priority
What breaks	When to fix

🎯 Senior-level line:

Severity reflects impact; priority reflects urgency.

6️⃣ Defect Triage (Interview Favorite)

What is Defect Triage?

A meeting/process where:

- Defects are reviewed
- Priority is assigned
- Fix timelines decided

Participants:

- QA
- Dev
- Product Owner
- Lead/Manager

Triage Outcomes

- Fix now
- Fix later
- Defer
- Reject
- Duplicate

7️⃣ Defect Rejection & How to Handle It

Common reasons:

- Not reproducible
- Works as designed
- Duplicate
- Environment issue

Best response:

Provide clear reproduction steps and evidence, and discuss constructively.

Never argue emotionally.

8️⃣ Defect Leakage (Important Metric)

Defect leakage occurs when:

- Defects escape to later testing stages or production

Example:

- Missed in system testing, found in UAT

Good line:

Defect leakage indicates gaps in test coverage.

9️⃣ Metrics in Defect Management

Common metrics:

- Defect density
- Defect rejection rate
- Defect leakage
- Defect aging

Interview tip:

Metrics are used for process improvement, not blame.

🔟 Defect Management in Agile

- Defects treated as backlog items
- Fixed within sprint if possible
- Continuous triage

1️⃣1️⃣ Mini Summary

Defect management ensures visibility, prioritization, and resolution of issues through a structured life cycle and collaborative triage.

CLUSTER 10: Test Closure, Metrics & Release Decision (Final Cluster)

This is the phase that proves **testing is complete, not abandoned**.

1️⃣ What is Test Closure?

Test Closure is the final phase of STLC where testing activities are:

- Formally evaluated
- Documented
- Signed off

🎯 Interview-grade line:

Test closure ensures testing is completed in a controlled and measurable manner.

2️⃣ Why Test Closure is Important

Without test closure:

- No clarity on test coverage
- No lessons learned
- No quality baseline for future projects

With test closure:

- Clear release confidence
- Metrics-driven decisions
- Process improvement insights

Senior framing:

Test closure transforms testing results into organizational knowledge.

3️⃣ Activities Performed During Test Closure

During test closure, QA performs:

- Test execution summary
- Defect analysis
- Coverage confirmation

- Risk assessment
- Lessons learned documentation
- Formal sign-off

4️⃣ Test Closure Report (Key Deliverable)

What is a Test Closure Report?

A document summarizing:

- What was tested
- What was not tested
- Quality status
- Known risks
- Release recommendation

Typical Contents of Test Closure Report

Section	Description
Scope Summary	Features tested
Test Execution Stats	Pass/Fail counts
Defect Summary	Open/Closed defects
Coverage	Requirement coverage
Risks	Known issues
Metrics	Quality indicators
Sign-off	Approval

5️⃣ Test Metrics (Very Important for Interviews)

Common Testing Metrics

Metric	Purpose
Test Case Coverage	Measure requirement coverage
Pass/Fail Percentage	Execution quality
Defect Density	Defects per module
Defect Leakage	Missed defects
Defect Aging	Resolution efficiency

🎯 Interview-safe line:

Metrics help assess process effectiveness, not individual performance.

6️⃣ Entry & Exit Criteria – Test Closure

Entry Criteria

- All planned tests executed
- Critical defects addressed
- Regression completed

Exit Criteria

- Closure report prepared
- Stakeholder sign-off obtained
- Artifacts archived

7️⃣ Release Decision (QA's Role)

QA provides:

- Quality assessment
- Risk-based recommendation

Important:

QA does not decide release alone but provides quality confidence.

You can say:

Release decisions are business-driven, supported by QA insights.

8️⃣ Handling Open Defects at Closure (Tricky Question)

If defects are still open:

- Assess severity & impact
- Get business sign-off
- Document risks clearly

Interview framing:

Low-risk defects may be deferred with stakeholder approval.

9️⃣ Test Closure in Agile

- Closure happens per sprint or release
- Retrospective captures lessons
- Metrics reviewed frequently

🔟 Final Summary of Chapter 2 (Use this verbatim if needed)

SDLC defines how software is built, while STLC defines how quality is ensured. Testing activities begin early, continue throughout development, and conclude with structured closure to provide release confidence and process improvement.

📌 CASE STUDY (PRACTICE ONLY)

CHUNK 1: Business Context + Wall of Words Requirement (Jira Style)

Project Name

SmartPay – Digital Wallet Web Application

Business Context (Background)

SmartPay is a digital wallet application that allows registered users to manage their wallet balance, link bank accounts, and perform peer-to-peer transfers. The business wants to introduce a **"Wallet Balance & Transaction History" feature** to improve user transparency and reduce customer support calls related to balance disputes.

📜 Wall of Words Requirement (As Seen in Jira)

Users should be able to view their current wallet balance once they log in to the SmartPay application. The balance shown should reflect the latest available balance after considering all completed transactions. Pending or failed transactions should not affect the displayed balance. However, users should be able to see pending transactions separately in the transaction history section.

The transaction history should display all wallet-related transactions such as wallet top-ups, peer-to-peer transfers sent, peer-to-peer transfers received, and refunds. Transactions should be displayed in reverse chronological order so that the most recent transactions appear first. Each transaction entry should include basic details like transaction date, transaction type, amount, and transaction status.

Users should be able to filter transaction history based on a selected date range. The maximum date range allowed for filtering should be three months at a time. If the selected date range exceeds the allowed limit, the system should display an appropriate message and should not load the data.

The system should support pagination for transaction history to avoid performance issues when loading large data sets. The default number of transactions displayed per page is expected to be around 20, but this value may be configurable later.

Transaction data displayed to the user should be accurate and consistent across sessions. Any discrepancies in balance or transaction history should be logged for audit and investigation purposes. Only completed and successful transactions should be considered for balance calculation.

The wallet balance and transaction history should be accessible on both desktop and mobile browsers. The page should load within acceptable performance limits even during peak usage times.

Acceptance Criteria (Loosely Defined – Typical Jira)

- User can view wallet balance
- Transaction history is displayed
- Filters work correctly
- Balance is accurate
- Page performance is acceptable

⚠️ **Intentionally Left Ambiguous (For You to Catch Later)**

- What defines “acceptable performance”?
- How are refunds treated?
- Time zone handling for transaction date?
- Pagination exact behavior?
- What is “pending” vs “failed”?

CHUNK 2: Business Requirements Brief (BRB Format)

Document Information

Item	Details
Document Name	BRB – Wallet Balance & Transaction History
Project	SmartPay Digital Wallet
Version	1.0
Prepared By	Business Team

Target Platform	Web (Desktop & Mobile)

---

1. Business Objective

Provide SmartPay users with **clear visibility into their wallet balance and transaction history**, improving transparency, trust, and self-service while ensuring accuracy and performance.

---

2. In-Scope

ID	In-Scope Item
IS-01	Display current wallet balance
IS-02	Display transaction history
IS-03	Filter transaction history by date
IS-04	Pagination of transaction records
IS-05	Audit logging for balance discrepancies

---

3. Out-of-Scope

ID	Out-of-Scope Item
OS-01	Transaction initiation (send money)
OS-02	Wallet top-up
OS-03	Statement download

---

4. Business Rules

Rule ID	Business Rule
BR-01	Wallet balance must reflect only completed and successful transactions
BR-02	Pending and failed transactions must not affect wallet balance
BR-03	Transaction history must show all wallet-related transactions
BR-04	Transactions must be displayed in reverse chronological order
BR-05	Maximum date range for filtering is 3 months
BR-06	Default pagination size is 20 records per page
BR-07	Balance discrepancies must be logged for audit

---

5. Functional Requirements

FR ID	Requirement Description
FR-01	System shall display current wallet balance after login
FR-02	System shall display transaction history
FR-03	System shall allow users to filter transactions by date range
FR-04	System shall restrict date range filter to maximum of 3 months
FR-05	System shall support pagination of transaction history
FR-06	System shall log balance discrepancies

---

6. Non-Functional Requirements

NFR ID	Requirement
NFR-01	Page load time shall be within 3 seconds under normal load
NFR-02	System shall support up to 10,000 concurrent users
NFR-03	Data displayed must be consistent across sessions
NFR-04	Application shall support major desktop and mobile browsers

---

7. User Flow (High Level)

Step	Description
1	User logs in
2	User navigates to Wallet page
3	System displays wallet balance
4	System displays transaction history
5	User applies filters or pagination

---

8. Assumptions & Dependencies

ID	Description
AD-01	User is authenticated
AD-02	Transaction data is available from backend
AD-03	System time zone is consistent

---

9. Acceptance Criteria

AC ID	Criteria
AC-01	Wallet balance is displayed correctly

AC ID	Criteria
AC-02	Transaction history loads successfully
AC-03	Date filter works within allowed range
AC-04	Pagination works correctly
AC-05	Audit logs are generated when required

CHUNK 3: Design Completion Artifacts (Functional Design View)

1. UI Design Overview (Textual Description)

Wallet Overview Page

The Wallet Overview page is accessible after successful login and contains two primary sections:

1. **Wallet Balance Section (Top Panel)**
2. **Transaction History Section (Below Balance Panel)**

The page is designed to be responsive for both desktop and mobile browsers.

---

2. Wallet Balance Section – Design Details

Display Elements

Field Name	Description
Wallet Balance Label	Static text “Wallet Balance”
Balance Amount	Displays currency and amount
Currency Code	Displayed as per user profile
Last Updated Timestamp	Shows last successful balance update

Design Constraints

- Balance amount is read-only
- Balance should update only after successful transactions
- Pending or failed transactions must not change displayed balance

---

3. Transaction History Section – Design Details

Transaction List Columns

Column Name	Description
Transaction Date	Date & time of transaction
Transaction Type	Credit / Debit
Description	Transfer Sent / Received / Refund / Top-up
Amount	Transaction amount with sign
Status	Completed / Pending / Failed

---

4. Sorting & Ordering Rules

- Default sorting: Transaction Date (Descending)
- Latest transaction appears at the top
- Sorting is system-controlled (user cannot change sort order)

---

5. Date Range Filter – Design

Filter Fields

Field	Description
From Date	Start date of filter
To Date	End date of filter
Apply Button	Applies date filter
Reset Button	Clears filter

Filter Constraints

- Maximum allowed range: 3 months
- From Date cannot be greater than To Date
- Error message displayed for invalid ranges
- No data load if validation fails

---

6. Pagination Design

Attribute	Value
Default Page Size	20 records
Navigation	Next / Previous
Page Number Display	Enabled
Load Behavior	Server-side pagination

---

7. Error & Message Handling (Design-Level)

Scenario	Message Behavior
No transactions available	Informational message displayed
Invalid date range	Validation error message
Backend failure	Generic error message
Data mismatch detected	Logged internally (no user display)

---

8. Audit & Logging (Design Completion Notes)

- Balance discrepancies logged with:
  - User ID
  - Timestamp
  - Expected balance
  - Actual balance
- Logs accessible only to authorized internal users

---

9. Non-Functional Design Constraints

Area	Constraint
Performance	Page load within 3 seconds
Security	No sensitive data in UI messages
Consistency	Same data across sessions
Compatibility	Desktop & mobile browsers

CHUNK 4: Change Request (CR) + Edge Conditions

This is the **final input** you would receive mid-project.

🔁 Change Request (CR)

CR ID: CR-WALLET-017

Change Description: Business has requested an enhancement to the **Wallet Balance & Transaction History** feature based on customer feedback and audit requirements.

Change Details

1. **Running Balance Requirement**
    - Transaction history should now display a **Running Balance** column.
    - Running Balance represents wallet balance **after each completed transaction**.
    - Running Balance should consider only **completed transactions**.
    - Pending and failed transactions should not impact running balance.
2. **Refund Visibility Update**
    - Refund transactions should be clearly labeled as **"Refund"** in the transaction description.
    - Refunds should increase wallet balance.
    - Refunds should be included in running balance calculation.
3. **Extended Date Filter**
    - Maximum date range for transaction history filtering is increased from **3 months to 6 months**.
    - Existing validation messages should be updated accordingly.
4. **Export Restriction (Partial Feature)**
    - Business discussed transaction export (CSV) but decided:
        - Export is **not enabled in this release**
        - UI should **not display any export option**

CR Scope Impact

Area	Impact
UI	New column added
Business Rules	Updated
Functional Requirements	Modified
RTM	Requires update
Test Cases	Requires regression
Non-Functional	No change

⚠️ Edge Conditions & Special Notes (Design + Business)

These were added informally via email/chat, not documented properly.

1. Running balance should:
    - Reset correctly after refunds
    - Handle multiple transactions with same timestamp
    - Handle zero-balance scenarios
2. Date filtering edge cases:
    - Leap year dates
    - Month-end boundaries
    - From Date equals To Date

3. Pagination behavior:
    - Running balance must remain accurate across pages
    - Page navigation should not recalculate values incorrectly
4. Time Zone Handling:
    - Transaction date should be displayed in **user's local time zone**
    - Backend provides timestamps in UTC
5. Performance Note:
    - Running balance calculation should not degrade page load time

---

🛑 FINAL STOP

You now have a **complete real-world case study**, including:

- Wall-of-words requirement
- BRB (structured)
- Design completion artifacts
- Change request
- Hidden edge conditions

🎯 Your practice goals:

- Impact analysis
- Update BRB / FRs / BRs
- Update RTM (multi-level)
- Identify regression scope
- Design new test scenarios & cases

QUESTIONS TO CHECK YOUR LEARNINGS

A. SDLC (Software Development Life Cycle)

SDLC Basics

1. What is SDLC?
2. Why is SDLC important?
3. What are the phases of SDLC?
4. Who is involved in each phase of SDLC?
5. Where does testing fit into SDLC?

SDLC Models

6. What are the different SDLC models?
7. Explain the Waterfall model.
8. What are the advantages and disadvantages of the Waterfall model?
9. Explain the V-Model.
10. How is testing mapped in the V-Model?
11. Difference between Waterfall and V-Model.
12. Explain the Spiral model.
13. When is the Spiral model preferred?
14. What is the Agile model?

15. Difference between Agile and Waterfall.
16. Can Agile work without documentation?
17. Which SDLC model did you work on in your project?
18. How do you choose an SDLC model for a project?

Testing in SDLC

19. At which SDLC phase does testing start?
20. Why is early testing important in SDLC?
21. What testing activities happen in the requirement phase?
22. What testing activities happen in the design phase?
23. What testing activities happen in the development phase?
24. What testing activities happen in the deployment phase?
25. What are the risks if testing is introduced late in SDLC?

B. STLC (Software Testing Life Cycle)

STLC Basics

26. What is STLC?
27. Why do we need STLC?
28. What are the phases of STLC?
29. Difference between SDLC and STLC.
30. How is STLC different from testing itself?

STLC Phases – Deep Dive

Requirement Analysis

31. What happens in the requirement analysis phase of STLC?
32. How do testers analyze requirements?
33. What is RTM?
34. Why is RTM important?
35. What challenges do testers face during requirement analysis?

Test Planning

36. What is a Test Plan?
37. Who prepares the Test Plan?
38. What are the contents of a Test Plan?
39. Difference between Test Plan and Test Strategy.
40. Can testing start without a Test Plan?

Test Case Design

41. What is test case design?
42. What inputs are needed to write test cases?
43. What is a test scenario?
44. Difference between test scenario and test case.
45. How do you ensure test case coverage?
46. What is traceability in test design?

Test Environment Setup

47. What is test environment?
48. Who is responsible for test environment setup?
49. What are common issues in test environment?
50. Can testing proceed without a stable environment?

Test Execution

51. What happens during test execution?
52. What is the difference between retesting and regression testing?
53. What documents are updated during test execution?
54. What do you do if a critical defect is found?
55. Can test execution start before development is complete?

Defect Reporting & Tracking

56. What is defect tracking?
57. What is a defect life cycle?
58. What is defect leakage?
59. What is defect triage?
60. Who participates in defect triage meetings?

Test Closure

61. What is test closure?
62. What activities are performed during test closure?
63. What is a test closure report?
64. Why is test closure important?
65. What metrics are collected during test closure?

C. SDLC vs STLC (Comparison Questions)

66. Difference between SDLC and STLC.
67. Can STLC exist without SDLC?
68. How are SDLC and STLC linked?
69. Which is broader: SDLC or STLC?
70. Where do Agile practices fit in SDLC & STLC?

D. Scenario-Based Questions (Very Common)

71. What if requirements keep changing during testing?
72. What if development is delayed but testing deadlines remain the same?
73. What if testers receive incomplete requirements?
74. How do you handle testing in a short release cycle?
75. What if exit criteria is not met but release date is fixed?

Chapter 3 – Test Design Techniques & Test Case Development

Segment 3.1 – Introduction to Test Design

3.1.1 What is Test Design?

Test Design is the process of **transforming software requirements into test artifacts** such as:

- Test scenarios
- Test cases
- Test conditions
- Test data

In simple terms:

Test design answers the question:
"What should be tested and how should it be tested?"

It acts as a **bridge between requirements and test execution**.

3.1.2 Why Test Design is Important

Test design is critical because:

- It ensures **requirement coverage**
- It avoids **missing critical scenarios**
- It reduces **redundant test cases**
- It improves **defect detection effectiveness**
- It enables **repeatable and auditable testing**

Poor test design results in:

- Production defects
- Test leakage
- Increased rework
- Loss of stakeholder confidence

3.1.3 Objectives of Test Design

The main objectives are:

1. Identify **all possible test conditions**
2. Cover both **functional and negative scenarios**
3. Optimize test effort with **maximum coverage**
4. Detect defects **as early as possible**
5. Provide **traceability** between requirements and tests

3.1.4 Test Design in STLC

Test design primarily occurs in the **Test Case Design phase** of STLC but **starts much earlier**:

STLC Phase	Test Design Activity
Requirement Analysis	Identify test scenarios
Test Planning	Decide design techniques
Test Case Design	Write test cases
Execution	Validate test design quality

Early involvement improves quality and reduces cost.

3.1.5 Relationship Between Requirements and Test Design

- Requirements define **what the system should do**
- Test design defines **how we verify that behavior**

Each requirement must be:

- Understood
- Broken into scenarios
- Covered by one or more test cases

This relationship is maintained using **traceability (RTM)**.

3.1.6 Risks of Poor Test Design

- Incomplete test coverage
- Missing boundary and negative cases
- Duplicate test cases
- Late defect detection
- Increased production issues

Interview insight: Most production defects are due to **poor test design**, not poor execution.

Segment 3.2 – Test Scenarios

3.2.1 What is a Test Scenario?

A **Test Scenario** is a **high-level description of what needs to be tested**.

It represents a **user action or system behavior** derived from requirements.

Example:

“Verify user login functionality”

Test scenarios answer:

“What functionality should be tested?”

3.2.2 Purpose of Test Scenarios

Test scenarios help to:

- Understand application behavior
- Ensure **complete requirement coverage**
- Serve as a base for writing test cases
- Communicate testing scope to stakeholders

They provide a **bird’s-eye view** of testing.

3.2.3 Characteristics of a Good Test Scenario

A good test scenario:

- Is requirement-based
- Is high-level
- Covers one business functionality
- Is easy to understand
- Is not dependent on test data

3.2.4 How to Identify Test Scenarios

Test scenarios are identified by:

- Reading functional requirements
- Understanding user flows
- Reviewing business use cases
- Analyzing system behavior

Each requirement usually maps to:

- One or more test scenarios

3.2.5 Business Scenario vs Test Scenario

Business Scenario	Test Scenario
Describes business workflow	Describes testable functionality
Written by business	Written by tester
Non-technical	Testing-oriented

Example:

- Business: “User purchases a product”
- Test Scenario: “Verify product checkout flow”

3.2.6 Advantages and Limitations

Advantages

- Easy to create
- High-level coverage
- Useful in early phases

Limitations

- Cannot be executed
- Do not contain steps
- Need conversion to test cases

Segment 3.3 – Test Cases

3.3.1 What is a Test Case?

A **Test Case** is a **set of conditions, inputs, actions, and expected results** to verify a specific functionality.

It answers:

“How exactly should the test be performed?”

3.3.2 Purpose of Test Cases

Test cases are used to:

- Validate requirements
- Detect defects
- Ensure repeatability
- Provide evidence of testing
- Support regression testing

3.3.3 Components of a Test Case

A standard test case includes:

- Test Case ID
- Test Case Description
- Preconditions
- Test Steps
- Test Data
- Expected Result
- Actual Result
- Status
- Comments

3.3.4 Characteristics of a Good Test Case

A good test case is:

- Clear and concise
- Independent
- Repeatable
- Maintainable
- Traceable to requirements

3.3.5 Positive, Negative & Edge Test Cases

- **Positive Test Cases**
  Validate expected behavior with valid inputs.
- **Negative Test Cases**
  Validate system behavior with invalid inputs.
- **Edge Test Cases**
  Validate behavior at boundary limits.

All three are essential for quality testing.

3.3.6 Reusable and Maintainable Test Cases

Good test cases:

- Avoid hardcoded values
- Use generic steps
- Can be reused across builds
- Require minimal updates

Segment 3.4 – Test Scenarios vs Test Cases

3.4.1 Conceptual Difference

Test Scenario	Test Case
What to test	How to test
High-level	Detailed
Non-executable	Executable
Broad coverage	Specific validation

3.4.2 Practical Difference

- Scenarios help **identify scope**
- Test cases help **execute tests**
- Scenarios come first
- Test cases are derived from scenarios

3.4.3 When to Focus More on Scenarios

- Early project phases
- Requirement reviews

- Test estimation
- High-level planning

3.4.4 When to Focus More on Test Cases

- During execution
- Regression testing
- Audit or compliance projects
- Detailed validation

3.4.5 Real-Time Example

Requirement: User login functionality

- Test Scenario:
  "Verify login functionality"
- Test Cases:
  - Login with valid credentials
  - Login with invalid password
  - Login with empty fields
  - Login with locked account

3.4.6 Interview Insight

A common interviewer trap: "Can we test without test cases?"

Correct thinking:

- You can **identify scenarios** without test cases
- You **cannot execute** without test cases

Segment 3.5 – Test Design Techniques (Overview)

3.5.1 Why Test Design Techniques Are Needed

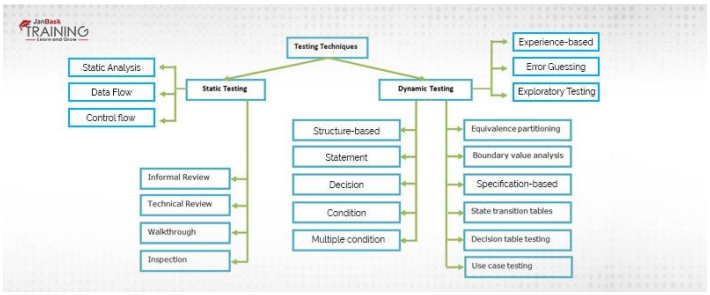
In real projects, requirements can generate **hundreds or thousands of possible test cases**. Testing everything is **not practical**, due to:

- Time constraints

- Cost limitations
- Resource availability
- Release pressure

Test design techniques help testers:

- Reduce the number of test cases
- Maintain adequate test coverage
- Detect defects effectively
- Avoid redundant testing

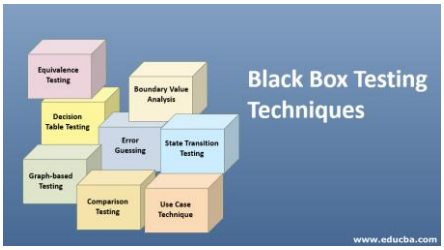


In short:

Test design techniques help test smarter, not harder.

3.5.2 What Are Test Design Techniques?

Test Design Techniques are **systematic methods** used to derive test cases from requirements in a structured and optimized manner.



They provide:

- Logical rules
- Clear boundaries
- Predictable coverage

Instead of random testing, techniques ensure **intentional testing**.

3.5.3 Classification of Test Design Techniques

Test design techniques are broadly classified into:

1. Black-Box Test Design Techniques

Focus on **functionality**, not internal code.

2. White-Box Test Design Techniques

Focus on **internal logic and code structure**
(usually done by developers or automation engineers)

3. Experience-Based Techniques

Based on tester's **experience and intuition**

👉 **Chapter 3 focuses mainly on Black-Box and Experience-Based techniques**, which are core to manual testing interviews.

3.5.4 Black-Box Test Design Techniques (Core Focus)

In black-box testing:

- Internal code is unknown
- Testing is done based on requirements
- User behavior is the primary concern

Common black-box techniques include:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Use Case Testing

Interview expectation: You should know **when to use which technique**, not just definitions.

3.5.5 Equivalence Partitioning (Preview)

Concept:

- Divide input data into groups
- Assume all values in a group behave the same

Purpose:

- Reduce test cases
- Maintain coverage

Example idea: If one value fails, others in the same group likely fail too.

(We will cover this in detail in Segment 3.6)

3.5.6 Boundary Value Analysis (Preview)

Concept:

- Defects often occur at boundaries
- Test values at the edges of valid ranges

Purpose:

- Catch off-by-one errors
- Validate limits

Example idea: Test minimum, maximum, just below, and just above limits. (Full depth in Segment 3.7)

3.5.7 Decision Table Testing (Preview)

Concept:

- Used when multiple conditions affect outcomes
- Helps test combinations logically

Purpose:

- Avoid missing condition combinations
- Handle complex business rules

Example idea: Login success depends on username validity AND password validity. (Detailed in Segment 3.8)

3.5.8 State Transition Testing (Preview)

Concept:

- System behavior depends on current state
- Same action may produce different results in different states

Purpose:

- Validate workflows
- Detect invalid transitions

Example idea: Login → Locked → Reset Password (Detailed in Segment 3.9)

3.5.9 Use Case Testing (Preview)

Concept:

- Test from end-user perspective
- Derived from business use cases

Purpose:

- Validate real-world workflows
- Ensure business flows work end-to-end

Example idea: Place order → Payment → Confirmation

(Detailed in Segment 3.10)

3.5.10 Experience-Based Techniques

These rely on:

- Tester's past experience
- Knowledge of common failures
- Understanding of application behavior

Most common: **Error Guessing**

Example:

- Empty fields
- Special characters
- Unexpected user actions

(Detailed in Segment 3.11)

3.5.11 Benefits of Using Test Design Techniques

Using proper techniques:

- Improves defect detection
- Ensures logical coverage
- Reduces duplicate test cases
- Saves time and effort
- Improves confidence in testing

Interview-ready statement: Test design techniques bring **discipline and predictability** to testing.

3.5.12 Limitations of Test Design Techniques

While powerful, they:

- Do not guarantee defect-free software
- Cannot replace domain knowledge
- Must be combined intelligently
- Depend on requirement clarity

Good testers: Combine **techniques + experience + business understanding**

3.5.13 Interview Perspective (Very Important)

Interviewers rarely ask:

- "Explain all techniques"

They usually ask:

- "How would you design test cases for this scenario?"
- "Which technique would you apply here and why?"
- "How do you reduce test cases without losing coverage?"

Your answer should include:

- Technique name
- Reason for choosing it
- Example input values

Segment 3.6 – Equivalence Partitioning

3.6.1 What is Equivalence Partitioning?

Equivalence Partitioning is a black-box test design technique in which:

- Input data is divided into **groups (partitions)**
- Each partition is assumed to behave **in the same way**
- One representative value from each partition is tested

Core idea:

If one value in a partition works, all other values in that partition are likely to work as well.

3.6.2 Why Equivalence Partitioning Is Used

Without EP:

- Test cases explode
- Redundant testing increases
- Time and effort are wasted

With EP:

- Fewer test cases
- Logical coverage
- Efficient testing

Interview insight:

EP is mainly used to **reduce test cases without reducing coverage**.

3.6.3 Types of Equivalence Classes

Equivalence classes are divided into:

1. Valid Equivalence Class

- Inputs that satisfy requirements
- System should **accept** these values

2. Invalid Equivalence Class

- Inputs that violate requirements
- System should **reject** these values

Every requirement must have:

- At least one valid class
- One or more invalid classes

3.6.4 Rules for Identifying Equivalence Classes

Common rules:

- Range-based inputs → create partitions
- Mandatory fields → valid vs empty
- Fixed values → matching vs non-matching
- Formats → correct vs incorrect format

Always ask:

“How can this input be valid?”
“How can this input be invalid?”

3.6.5 Step-by-Step Example (Classic Interview Case)

Requirement
Age must be between **18 and 60**

Step 1: Identify partitions

- Valid:
  - 18 to 60
- Invalid:
  - Less than 18
  - Greater than 60

Step 2: Select representative values

- Valid: 30
- Invalid (low): 15
- Invalid (high): 65

👉 Only **3 test cases**, instead of testing all ages.

3.6.6 Where Equivalence Partitioning Is Best Used

- Input fields
- Forms
- Numeric ranges
- Drop-down values
- Validation rules

3.6.7 Advantages of Equivalence Partitioning

- Reduces test cases
- Saves time
- Simple to apply
- Good for large input domains

3.6.8 Limitations of Equivalence Partitioning

- Does not focus on boundary defects

- Assumes uniform behavior within partition
- Needs to be combined with BVA

Interview-ready line:

Equivalence Partitioning identifies **what to test**, but not **where defects usually hide**.

Segment 3.7 – Boundary Value Analysis (BVA)

3.7.1 What is Boundary Value Analysis?

Boundary Value Analysis is a black-box test design technique that focuses on:

- Testing values at the **edges of input ranges**

Principle:

Defects are more likely to occur at boundaries than at the center.

3.7.2 Why Boundaries Are Important

Common boundary defects:

- Off-by-one errors
- Incorrect comparison operators
- Incorrect limits

Examples:

- < instead of <=
- > instead of >=

3.7.3 Boundary Values Explained

For a range **X to Y**, BVA focuses on:

- X − 1 (just below minimum)
- X (minimum)
- X + 1 (just above minimum)
- Y − 1 (just below maximum)
- Y (maximum)

- Y + 1 (just above maximum)

These values expose boundary-related defects.

3.7.4 Step-by-Step Example (Same Requirement)

Requirement
Age must be between **18 and 60**

Boundary values:

- 17 (below min)
- 18 (min)
- 19 (above min)
- 59 (below max)
- 60 (max)
- 61 (above max)

👉 These values specifically test the **edges**.

3.7.5 Relationship Between EP and BVA

Aspect	Equivalence Partitioning	Boundary Value Analysis
Focus	Input groups	Input edges
Goal	Reduce test cases	Catch boundary defects
Coverage	Broad	Deep
Used for	Valid/invalid inputs	Limits and edges

Interview gold line:

BVA is applied **on top of** equivalence partitioning.

3.7.6 When to Use BVA

- Numeric ranges
- Length validations
- Date ranges
- Amount limits
- Quantity fields

3.7.7 Advantages of BVA

- High defect detection rate
- Catches critical defects
- Simple and effective

3.7.8 Limitations of BVA

- Not useful for non-ordered values
- Cannot test internal logic
- Needs clear boundaries

3.7.9 Combined Use of EP and BVA (Real Testing Practice)

Good testers:

1. Identify equivalence classes
2. Apply BVA on boundaries of valid classes
3. Add negative cases
4. Cover edge cases

This combination gives:

- Optimized test count
- Maximum coverage
- High confidence

3.7.10 Interview Perspective (VERY IMPORTANT)

Interviewers may ask:

- “Which technique would you apply here?”
- “How many test cases will you design?”
- “Why not test all values?”

Correct approach:

- Mention EP to reduce cases
- Mention BVA to catch boundary defects
- Give example values

Segment 3.8 – Decision Table Testing

3.8.1 What is Decision Table Testing?

Decision Table Testing is a black-box test design technique used when:

- System behavior depends on **multiple conditions**
- Different combinations of conditions produce different outcomes

It represents logic in a **tabular format**, making complex rules clear and testable.

Core idea:

When multiple inputs interact, test **combinations**, not isolated values.

3.8.2 When to Use Decision Table Testing

Use this technique when:

- Business rules are complex
- Multiple conditions must be satisfied
- Rules are interdependent
- Requirements contain “AND / OR” logic

Typical use cases:

- Login rules
- Discounts and offers
- Approval workflows
- Eligibility checks

3.8.3 Components of a Decision Table

A decision table consists of:

1. **Conditions**
   Inputs or system states
2. **Actions**
   Expected outcomes
3. **Rules**
   Combinations of conditions leading to actions

3.8.4 Structure of a Decision Table

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Condition A	Y	Y	N	N
Condition B	Y	N	Y	N
Action	Allow	Deny	Deny	Deny

Each column represents **one test case**.

3.8.5 Step-by-Step Example (Interview Favorite)

Requirement
Login succeeds only if:

- Username is valid
- Password is valid

Step 1: Identify conditions

- Username valid? (Yes/No)
- Password valid? (Yes/No)

Step 2: Identify actions

- Login successful
- Error message displayed

Step 3: Create decision table

Username	Password	Result
Valid	Valid	Login success
Valid	Invalid	Error
Invalid	Valid	Error
Invalid	Invalid	Error

👉 Four combinations = four test cases

3.8.6 Benefits of Decision Table Testing

- Ensures no combination is missed
- Simplifies complex logic
- Improves coverage
- Easy to review

3.8.7 Limitations of Decision Table Testing

- Number of combinations increases quickly
- Not ideal for simple validations
- Requires clear business rules

Interview insight:

Decision tables convert **confusing logic into visible logic**.

Segment 3.9 – State Transition Testing

3.9.1 What is State Transition Testing?

State Transition Testing is used when:

- System behavior depends on **current state**
- Same action gives different results in different states

A **state** represents the system condition at a point in time.

3.9.2 Key Terminology

- **State**: Current condition of the system
- **Event**: Action that causes change
- **Transition**: Movement from one state to another
- **Valid Transition**: Allowed movement
- **Invalid Transition**: Disallowed movement

3.9.3 When to Use State Transition Testing

Used when:

- Workflow based systems exist
- History matters
- Status changes occur

Common examples:

- Login attempts
- Order processing
- Ticket lifecycle
- Approval systems

3.9.4 Step-by-Step Example (Very Common Interview Case)

Requirement
User account states:

- Active
- Locked

Rules:

- After 3 invalid login attempts → Account locked
- Locked user cannot login

States and transitions:

Current State	Event	Next State
Active	Invalid login (1–2 times)	Active
Active	Invalid login (3rd time)	Locked
Locked	Login attempt	Locked

3.9.5 State Transition Diagram (Conceptual)

- Start in **Active**
- Invalid login events increase counter
- On 3rd failure → transition to **Locked**
- No transition back unless reset

This diagram helps visualize **allowed and disallowed paths**.

3.9.6 Test Case Derivation from State Transitions

Test cases should cover:

- All valid transitions
- All invalid transitions
- Repeated events
- Boundary conditions (e.g., 3rd attempt)

3.9.7 Benefits of State Transition Testing

- Ensures workflow correctness
- Detects missing validations
- Useful for complex systems
- Prevents illegal state changes

3.9.8 Limitations of State Transition Testing

- Needs clear state definitions
- Not suitable for stateless systems
- Can become complex with many states

3.9.9 Decision Table vs State Transition Testing

Aspect	Decision Table	State Transition
Focus	Conditions & rules	States & behavior
Best for	Logical combinations	Workflows
Example	Discount rules	Login lockout

Interview gold:

Use decision tables for **rules**, state transitions for **behavior over time**.

3.9.10 Interview Perspective (Important)

Interviewers may ask:

- Which technique would you use here?
- How many test cases will you design?
- How will you handle invalid transitions?

Strong answer pattern:

1. Identify technique
2. Explain why
3. Give a simple example

Segment 3.10 – Use Case Testing

3.10.1 What is Use Case Testing?

Use Case Testing is a black-box test design technique where:

- Test cases are derived from **use cases**
- Focus is on **end-to-end user interactions**
- Testing is done from a **real user perspective**

A **use case** describes how a user interacts with the system to achieve a goal.

Core idea:

Test the system the way users actually use it, not the way developers think it works.

3.10.2 What is a Use Case?

A use case typically contains:

- Actor (user/system)
- Precondition
- Main flow (happy path)
- Alternate flows
- Exception flows
- Postcondition

Example: “User places an order successfully”

3.10.3 Main Flow, Alternate Flow, Exception Flow

These three flows are **very important in interviews**.

Main Flow (Happy Path)

- Normal, expected sequence
- No errors
- Everything works as intended

Example: Valid user → adds product → pays → order confirmed

Alternate Flow

- Valid variations
- Still acceptable behavior

Example:

User changes address before payment

Exception Flow

- Error or failure scenarios
- System deviates due to invalid action or issue

Example: Payment failure → error message → retry option

3.10.4 How to Derive Test Scenarios from Use Cases

Steps:

1. Read the complete use case
2. Identify user goals
3. Identify all flows
4. Convert each flow into a test scenario

Each flow = at least one test scenario.

3.10.5 How to Derive Test Cases from Use Cases

From each scenario:

- Break into steps
- Identify inputs
- Define expected results
- Include validations at each step

This creates **end-to-end test cases**.

3.10.6 Use Case Testing vs Functional Testing

Aspect	Use Case Testing	Functional Testing
Focus	User journey	Individual function
Scope	End-to-end	Feature-level
Perspective	Business/user	System
Coverage	Broad	Deep

Interview-ready line: Use case testing ensures business flows work seamlessly across features.

3.10.7 Advantages of Use Case Testing

- Mimics real user behavior
- Covers integration points
- Improves business confidence
- Useful in UAT

3.10.8 Limitations of Use Case Testing

- May miss internal edge cases
- Depends on quality of use cases
- Needs to be combined with other techniques

Segment 3.11 – Error Guessing

3.11.1 What is Error Guessing?

Error Guessing is an experience-based test design technique where:

- Test cases are designed based on tester's intuition
- Past defect knowledge is applied
- No formal rules are followed

Core idea: "Where can this system possibly break?"

3.11.2 Why Error Guessing Is Important

Not all defects:

- Follow rules
- Occur at boundaries
- Fit into partitions

Some defects appear due to:

- User behavior
- Environment issues
- Integration gaps
- Oversights

Error guessing fills these gaps.

3.11.3 Who Performs Error Guessing?

- Experienced testers
- Domain experts
- Senior QA members

However: Even junior testers can apply basic error guessing using common sense.

3.11.4 Common Error Guessing Patterns

Typical areas testers explore:

- Empty inputs
- Special characters
- Maximum field lengths
- Rapid repeated actions
- Browser refresh during transactions
- Session timeout scenarios
- Invalid file uploads
- Network interruption cases

3.11.5 Real-Time Example

Login page

- Copy-paste password
- Leading/trailing spaces
- Browser back button after logout
- Multiple tabs logged in
- Refresh after login

These are rarely written in requirements but often cause bugs.

3.11.6 Advantages of Error Guessing

- Finds hidden defects
- Mimics real user behavior
- Improves product robustness
- Adds tester value

3.11.7 Risks and Limitations of Error Guessing

- Not structured
- Depends on tester skill
- Cannot guarantee coverage
- Hard to measure completeness

Interview insight:

Error guessing should **complement**, not replace, formal techniques.

3.11.8 Error Guessing vs Structured Techniques

Aspect	Structured Techniques	Error Guessing
Basis	Rules & logic	Experience
Coverage	Predictable	Exploratory
Repeatability	High	Low
Defect type	Known patterns	Unknown patterns

3.11.9 Interview Perspective (Very Common Question)

Question:
"How do you design test cases when requirements are incomplete?"

Strong answer includes:

- Use available requirements
- Apply test design techniques
- Add error guessing based on experience

This shows **balanced testing maturity**.

QUESTIONS TO CHECK YOUR LEARNINGS

A. Test Design Fundamentals

1. What is test design?
2. Why is test design important?
3. When does test design start in STLC?
4. What inputs are required for test design?
5. What is test coverage?
6. How do you ensure maximum test coverage?
7. What happens if test design is poor?

B. Test Scenarios vs Test Cases

8. What is a test scenario?
9. What is a test case?
10. Difference between test scenario and test case.
11. Which comes first: test scenario or test case?
12. Why do we need test scenarios?
13. Can test cases exist without test scenarios?
14. Give a real-time example of a test scenario and test case.

C. Test Case Structure & Writing

15. What are the components of a test case?
16. What is a good test case?
17. How do you write effective test cases?
18. What is precondition and postcondition?
19. What is expected result?
20. What is test data?
21. How do you write negative test cases?
22. What mistakes should be avoided while writing test cases?
23. How do you review test cases?

D. Test Design Techniques (Very High Frequency)

Equivalence Partitioning

24. What is equivalence partitioning?
25. Why is equivalence partitioning used?
26. How do you identify equivalence classes?
27. Give a real-time example of equivalence partitioning.
28. What are valid and invalid equivalence classes?

Boundary Value Analysis (BVA)

29. What is boundary value analysis?
30. Why are boundary values important?
31. Difference between equivalence partitioning and BVA.
32. Give a real-time example of BVA.
33. Which values are tested in BVA?

Decision Table Testing

34. What is decision table testing?
35. When is decision table testing used?
36. What are conditions and actions in a decision table?
37. Give an example where decision table testing is useful.
38. What problems does decision table testing solve?

State Transition Testing

39. What is state transition testing?

40. What is a state?
41. What is a transition?
42. What is an invalid transition?
43. Give a real-time example of state transition testing.
44. Where is state transition testing commonly used?

Use Case Testing

45. What is use case testing?
46. What is a use case?
47. How do you derive test cases from use cases?
48. What is the difference between use case testing and functional testing?
49. What are main flow and alternate flow?

Error Guessing

50. What is error guessing?
51. Who performs error guessing?
52. Why is error guessing important?
53. Give examples of error guessing from your project.
54. Is error guessing a formal technique?

E. Positive & Negative Testing

55. What is positive testing?
56. What is negative testing?
57. Why is negative testing important?
58. Give examples of negative test cases.
59. Can an application pass positive testing and still fail in production?

F. Traceability & Coverage

60. What is traceability?
61. What is RTM?
62. How does RTM help in test design?
63. What is requirement coverage?
64. What happens if requirements are missed in RTM?

G. Scenario-Based Questions (Very Common)

65. How do you design test cases when requirements are unclear?
66. How do you prioritize test cases?
67. What do you do if time is limited but test cases are many?
68. How do you ensure critical functionality is tested first?
69. How do you handle frequent requirement changes during test design?
70. How do you design test cases for a complex module?